

SVKM's NMIMS University

(Deemed-to-be University)

Agentic Legacy Modernization System (ALMS)

Project Report submitted in partial fulfilment

of

(B.Tech)

In

(Computer Science & Business Systems)

by

Devang Gajre (E011)

Padmanabh Kakad (E025)

Krish Sarkar (E055)

Mithil Save (E056)

Under the supervision of

Rupesh Mishra

(Department of Computer Science & Business Systems, MPSTME)

SVKM's NMIMS University

(Deemed-to-be University)

**MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT &
ENGINEERING (MPSTME)**

Vile Parle (W), Mumbai-56

(2026–2027)

Certificate

CERTIFICATE

This is to certify that the project entitled “**Agentic Legacy Modernization System (ALMS)**” has been carried out by the following students: Devang Gajre (E011), Padmanabh Kakad (E025), Krish Sarkar (E055), and Mithil Save (E056), under my guidance and supervision, and has been submitted in partial fulfilment of the degree of B.Tech in Computer Science & Business Systems of MPSTME, SVKM’s NMIMS (Deemed-to-be University), Mumbai, India.

Project Mentor (Name and Signature)

Examiner (Name and Signature)
(Internal Guide)

Date: _____

Place: Mumbai

(HoD) (Name and Signature)

Acknowledgement

We would like to express our sincere gratitude to our project mentor, Rupesh Mishra, for his continuous guidance, patience, and constructive feedback throughout the duration of this project. His guidance on the system’s technical direction, and his insistence on candor in reporting results, shaped the outcome presented in this document.

We are also grateful to the Mukesh Patel School of Technology Management & Engineering (MPSTME), SVKM’s NMIMS University, for providing the academic environment, coursework foundation, and infrastructure that made this capstone project possible. We thank the Department of Computer Science & Business Systems for the opportunity to pursue a project of this scope as part of our B.Tech curriculum.

Finally, we acknowledge the open-source communities behind the tools this project depends on - LangGraph, Ollama, ChromaDB, python-louvain, FastAPI, and pytest/hypothesis among others - without which a fully local, zero-cost pipeline of this kind would not have been practical to build within an academic timeline.

Name	Roll No.	SAP ID
Devang Gajre	E011	70362300082
Padmanabh Kakad	E025	70362300066
Krish Sarkar	E055	70362300062
Mithil Save	E056	70362300081

Abstract

Migrating a monolithic application to a microservice architecture is one of the most disruptive undertakings a software team can perform, and the research literature confirms what practitioners already know from experience: such migrations are largely manual, span months, and depend heavily on an individual engineer’s judgement of where the architectural seams lie. Automated pipelines built on large language models (LLMs) have recently pushed further into this problem than static or trace-based techniques alone, proposing service boundaries directly from source code and, in the most advanced cases, carrying a migration all the way to a deployable cloud artifact. However, the pipelines that reach furthest down the migration path do so by depending on hosted, commercial-scale models and cloud inference, or on GPU fine-tuning of embedding models - design choices that exclude students, independent developers without a cloud budget or GPU access, and organizations whose source code cannot leave their own infrastructure for privacy or regulatory reasons.

The Agentic Legacy Modernization System (ALMS) is a multi-agent pipeline that migrates a Python monolith into a set of containerised FastAPI microservices on a single commodity laptop, using a local, open-weight language model served by Ollama, at zero API cost, and without a GPU. Its design keeps everything deterministic outside the language model and gives the model only the one transformation step it is suited to. Service boundary identification is a graph-clustering problem: an AST-derived dependency graph, filtered of non-architectural edges (calls to Python builtins, standard-library methods, and local-variable methods), is reduced by seed-fixed Louvain community detection rather than by prompting a model. Output validation is a compiler problem: every generated service is gated by `py_compile` inside a bounded retry loop. Inter-agent communication runs on schema-validated data contracts, so no model judges another model’s output. The local model handles only code generation and test generation, both grounded in a local retrieval-augmented knowledge base of FastAPI, Domain-Driven Design, security, and testing pattern documents that fills in what a small local model may not know about current framework idioms.

ALMS runs on LangGraph for stateful orchestration, Ollama for local inference, ChromaDB for retrieval, and `python-louvain` for community detection. The evaluation reported here is a single-case feasibility study, not a benchmark. On a 1,578-line, six-module Flask monolith, deterministic structural filtering cuts the dependency graph used for boundary selection by 56.1% of its nodes and 61.0% of its edges before any clustering runs. Using a 30-billion-parameter Mixture-of-Experts local model

(qwen3-coder:30b, approximately 3.3 billion active parameters per token) served through CPU memory-mapped weight paging, a single unattended run completed the full decompose-generate-test-deploy pipeline end to end. All four proposed services compiled cleanly with no manual intervention; the pipeline generated 1,120 test cases across 570 files; three of the four resulting containers booted behind a working Nginx API gateway. The fourth did not: an orchestration-glue service with no owned data got clustered into its own service and failed to boot. Only 58.2% of the generated test files are valid, compilable Python, because test generation, unlike code generation, carries no compile-retry gate. Two services independently regenerated one another's CRUD logic, so declared data ownership is not yet enforced at generation time.

A disciplined deterministic/LLM split lets even a resource-constrained local model carry a full migration pipeline through to a mostly-bootable, containerised output, at zero cloud cost, on hardware a single developer already owns.

Contents

Abbreviations	ix
1 Introduction	1
1.1 Background of the Project Topic	1
1.2 Motivation and Scope of the Report	2
1.3 Problem Statement	2
1.4 Salient Contribution	3
1.5 Organization of the Report	4
2 Literature Survey	5
2.1 Introduction to the Overall Topic	5
2.2 Detailed Survey of Related Work	6
2.2.1 Systematic Reviews of Migration Research	6
2.2.2 Trace-Based and Graph-Based Decomposition	6
2.2.3 LLM-Based Decomposition and Migration	7
2.2.4 Local and Private LLM Deployment	8
2.3 Research Gap and Problem Definition	8
3 Methodology and Implementation	10
3.1 Block Diagram	10
3.2 Hardware and Software Environment	11
3.3 Software Description, Flowchart and Algorithm	12
3.3.1 Deterministic structural analysis	12
3.3.2 Service boundary identification	13
3.3.3 Retrieval-augmented generation	14
3.3.4 Code generation and the compile-retry loop	14
3.3.5 Test generation and shadow testing	15
3.3.6 Orchestration: fan-out, reducers, and human gates	16
3.3.7 Output and data contracts	16
4 Results and Analysis	17

4.1	Evaluation Methodology and Standards Adherence	17
4.2	Results	18
4.2.1	RQ1: Does the pipeline complete on commodity, CPU-only hardware?	18
4.2.2	RQ2: Is the generated output syntactically valid and bootable? .	18
4.2.3	RQ3: What effect does deterministic structural filtering have? .	19
4.2.4	RQ4: What is the resource and cost profile relative to a hosted pipeline?	19
4.3	Case Study Walkthrough	21
5	Advantages, Limitations and Applications	23
5.1	Advantages	23
5.2	Limitations	24
5.3	Applications	25
6	Conclusion and Future Scope	27
	References	29
I	Soft Code Flowcharts	32
II	Software and Hardware Environment Specifications	33
III	List of Software Components	34
IV	List of Papers Presented and Published	36

List of Figures

3.1	End-to-end ALMS migration pipeline: deterministic structural analysis and boundary identification feed a per-service fan-out in which a local LLM agent generates code (gated by a bounded compile-retry loop) and tests, checkpointed by three human-in-the-loop gates. All LLM stages draw on a local retrieval-augmented pattern knowledge base.	11
3.2	LangGraph per-service subgraph (<code>process_service</code>), run once per fan-out branch. Test generation is reached only on a compile pass and has no equivalent retry cycle of its own (Section 3.3.5, Section 4.2).	15
4.1	Dependency graph of <code>examples/sample_monolith</code> before and after deterministic filtering of non-architectural calls (Python builtins, <code>stdlib</code> methods, local-variable methods), recomputed directly from the current Analyzer implementation. Filtering reduces the graph from 171 nodes / 267 edges / 6 hotspots to 75 nodes / 104 edges / 2 hotspots (56.1% / 61.0% reduction).	20
I.1	Reproduction of Figure 3.1: end-to-end ALMS migration pipeline flowchart.	32
I.2	Reproduction of Figure 3.2: LangGraph per-service compile-retry sub-graph flowchart.	32

List of Tables

3.1 ALMS Software Environment and Technology Stack. 12

4.1 Resource and Cost Profile (sample_monolith, CPU-only). 20

4.2 Comparison of Monolith-to-Microservice Migration Systems. “Reach”
= Decompose → Generate → Test → Deploy. 21

III.1 Software Components and Libraries. 35

Abbreviations

ALMS	Agentic Legacy Modernization System
AST	Abstract Syntax Tree
API	Application Programming Interface
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
DDD	Domain-Driven Design
GPU	Graphics Processing Unit
HITL	Human-in-the-Loop
HTTP	Hypertext Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
KB	Knowledge Base
LLM	Large Language Model
LOC	Lines of Code
MoE	Mixture-of-Experts
ORM	Object-Relational Mapping
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
REST	Representational State Transfer
SQL	Structured Query Language
YAML	YAML Ain't Markup Language

Chapter 1

Introduction

This chapter introduces the Agentic Legacy Modernization System (ALMS), the project undertaken for this capstone report. It presents the background against which the project is situated, the motivation and scope of the work, a precise statement of the problem being addressed, the salient contributions of the project, and an outline of how the remainder of this report is organized.

1.1 Background of the Project Topic

A monolithic application is one in which all business functionality - user management, order processing, payments, and so on - lives in a single deployable codebase and a single process boundary. Monoliths are easy to build and reason about early in a project's life, but as a codebase grows they become progressively harder to scale, test, and deploy independently, which is why the microservice architectural style - decomposing an application into a set of small, independently deployable services, each owning its own data - has become the default target architecture for systems expected to scale in both traffic and team size.

The transformation from monolith to microservices is, however, one of the most disruptive undertakings a software team can perform. It requires identifying seams in a codebase that was never designed to be split, defining service boundaries along those seams, rewriting request handling and data access around the new boundaries, and re-establishing confidence, usually through extensive testing, that the new system behaves like the old one. Systematic literature reviews of the migration research area confirm what practitioners already know from direct experience: migration efforts are largely manual, span months, and depend heavily on the individual engineer's judgement of where the architectural seams lie [1, 2]. Boundary identification in particular is repeatedly singled out as the single hardest sub-problem in the area, and systematic surveys find that the majority of proposed academic approaches evaluate on a single system in a lab setting and stop short of producing anything a team could deploy [1, 2, 3].

Large language models (LLMs) have recently been applied to this problem with promising early results: LLM-driven pipelines can propose service boundaries directly from source code [4] and, in the most advanced published work, can carry a migration all the way through to a deployable cloud artifact [5]. The pipelines that reach fur-

thrust down the migration path, however, also carry the steepest resource requirements. Approaches that learn a boundary-selection embedding typically require fine-tuning a language model, which in practice means GPU time and a curated corpus of existing microservice repositories [4]. Pipelines that generate and deploy code typically orchestrate hosted, commercial-scale assistants, calling out to cloud APIs for every generation step [5]. Both design points assume an evaluator with a GPU budget, a cloud billing account, or both, and a willingness to send proprietary source code to a third-party API - an assumption that excludes a large and legitimate constituency of students, independent developers, small teams, and organizations operating under privacy or regulatory constraints [6].

1.2 Motivation and Scope of the Report

The motivation for this project is direct: to determine whether a full monolith-to-microservice migration pipeline - one that reaches all the way from source-code analysis to a running, containerised, tested deployment - can be built and executed using only a local, open-weight language model on hardware a single student or developer already owns, with no cloud inference, no GPU, and no per-run API cost. This question matters specifically because it has not been directly answered by prior work: as the literature survey in Chapter 2 documents, published systems either stop at the boundary-proposal stage without generating or deploying anything, or reach a deployable artifact only by depending on hosted, commercial-scale models.

The scope of this report is correspondingly a system-and-artifact report rather than a benchmark study. It documents the design, implementation, and a single, honest case-study evaluation of ALMS on one representative Python monolith. It does not claim that the reported results generalize to arbitrary codebases, larger systems, or other programming languages; Chapter 4 and Chapter 5 discuss these limitations explicitly rather than understating them. The scope also covers the eight-week development process by which the system was built, from initial literature review and mentor-approved roadmap through to a full end-to-end pipeline run, since the process itself - particularly the model-upgrade decision made partway through the project - is part of what this report evaluates.

1.3 Problem Statement

Existing automated approaches to monolith-to-microservice migration force a trade-off that this project set out to avoid. On one side, systems that rely on hosted, commercial-scale LLMs or GPU-fine-tuned embedding models reach further down the migration pipeline - proposing boundaries, generating code, and in some cases deploying the re-

sult - but require resources (cloud budgets, GPUs, willingness to transmit source code externally) that are unavailable or unacceptable to a large class of developers and organizations. On the other side, systems designed to run without those resources have, in the published literature, stopped at the class-to-service boundary-proposal stage and have not been shown to produce a running, deployable artifact.

The problem this project addresses is therefore: *can a monolith-to-microservice migration pipeline be designed so that a local, open-weight language model - running entirely offline, on commodity, CPU-only hardware, at zero API cost - carries the migration all the way from source-code analysis through service-boundary identification, code generation, test generation, and containerised deployment, while remaining honest about where such a pipeline currently falls short?* Answering this requires both a system design that minimises what is asked of the local model, and an evaluation methodology that reports negative results with the same rigour as positive ones.

1.4 Salient Contribution

This project makes the following contributions, each of which is elaborated in the corresponding later chapter:

1. **A deterministic/LLM split for local migration** (Chapter 3): a pipeline architecture in which service-boundary selection, output validation, and inter-agent data contracts are all handled deterministically by ordinary, checkable code, and the local language model is responsible only for the code-generation and test-generation steps it is suited to.
2. **A fully local, zero-cost, offline implementation** (Chapters 3–4): an end-to-end system spanning analysis, boundary proposal, code generation, test generation, and containerised deployment that runs on a CPU-only, RAM-constrained laptop with no calls to a hosted model provider, evaluated with a 30-billion-parameter Mixture-of-Experts model served through CPU memory-mapped weight paging.
3. **An eight-week, mentor-guided development process** (Chapter 3) that progressed from literature review and architectural design through to individual agent implementation, full LangGraph integration, and a documented mid-project model upgrade from a 7-billion-parameter to a 30-billion-parameter local model via a “layering” methodology, once the smaller model’s output quality proved insufficient.
4. **A single-case feasibility evaluation that reports its failures** (Chapters 4–5): all four proposed services compiled, the API gateway worked, three of four containers booted, and the dependency graph used for boundary selection shrank by

56.1%/61.0%. One non-domain service failed to start. Test-generation output validity (58.2%) lags code generation's (100%) because it lacks the same compile-retry discipline, and the current data contracts do not stop sibling services from duplicating each other's endpoints.

1.5 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 surveys fifteen related works in monolith decomposition, LLM-assisted migration, LLM-based test migration, and local/private LLM deployment, and uses this survey to identify the specific gap in the literature that ALMS occupies. Chapter 3 describes the ALMS system methodology and implementation: the overall architecture, each pipeline stage, the orchestration mechanism, the technology stack, and the eight-week process by which the system was built. Chapter 4 reports the case-study results and their analysis, organized around four research questions concerning feasibility, output validity, the effect of deterministic filtering, and resource cost. Chapter 5 discusses the advantages, limitations, and applications of the system candidly and in one place. Chapter 6 concludes the report and outlines the scope for future work. The report closes with a numbered reference list and four appendices covering flowcharts, environment specifications, software components, and publication status.

Chapter 2

Literature Survey

This chapter surveys the published research most directly relevant to ALMS, in order to establish what has already been achieved in automated monolith-to-microservice migration, where the reviewed approaches stop short, and precisely which gap in the literature this project’s design targets. The chapter proceeds by first situating the overall topic against fifteen references drawn from systematic reviews, tool papers, and recent LLM-based approaches (Section 2.1), then surveys ten of those works in depth, in the serial order in which they are cited (Section 2.2), before concluding with the specific research gap the survey identifies.

2.1 Introduction to the Overall Topic

Migrating a monolithic application to a microservice architecture has attracted a substantial and growing body of research attention over the past decade, driven by the practical difficulty organizations face when scaling monolithic systems. At least three independent systematic literature reviews published within the last three years survey this area from different corpora and methodologies, and reach a strikingly consistent conclusion: decomposition research is dominated by single-system, lab-only evaluations that classify modules into service groups and stop, rarely producing a running, deployable artifact [1, 2, 3]. This section draws on fifteen references spanning four related strands of work: (i) systematic reviews and mapping studies of the migration literature as a whole [1, 2, 3]; (ii) graph- and trace-based decomposition tools that predate the widespread use of LLMs [7, 8]; (iii) LLM-based decomposition, migration, and code-generation approaches, including several published in the last two years [4, 9, 10, 11, 5, 12, 13, 14]; and (iv) work on the practical and privacy considerations of deploying language models locally rather than through a hosted API [6, 15]. Read together, these fifteen references let the survey below identify exactly where the state of the art currently stops, and hence what a new contribution in this space needs to do differently.

2.2 Detailed Survey of Related Work

2.2.1 Systematic Reviews of Migration Research

Hassan, Abdel-Fattah, and Mohamed [1] conduct a systematic literature review of monolith-to-microservice migration approaches published across the preceding decade, and find that boundary identification is consistently reported as the single hardest step in the migration process, ahead of code rewriting or data-layer separation. Martínez Saucedo et al. [2] perform a complementary systematic mapping study and report the same finding from a partially overlapping but distinct corpus, additionally observing that the decomposition literature skews heavily toward Java applications, leaving migration of Python monoliths comparatively under-served. Abgaz et al. [3], publishing in *IEEE Transactions on Software Engineering*, extend this picture with the most methodologically rigorous review of the three, classifying decomposition techniques by the artifact they analyse (static structure, dynamic traces, or a hybrid of both) and concluding that the great majority of proposed approaches are evaluated in a lab setting on a single system and do not report deployment outcomes. All three reviews independently converge on the same two gaps: migration research rarely reaches a deployable artifact, and it is rarely evaluated with the same rigour on more than one codebase.

2.2.2 Trace-Based and Graph-Based Decomposition

Kalia et al. [7] present Mono2Micro, the most widely cited practical decomposition tool in the runtime-trace family. Mono2Micro records execution traces labelled with business use cases, builds a class-similarity measure from direct and indirect call patterns observed across those traces, and applies hierarchical agglomerative clustering to produce an explainable class-to-service partition. It was evaluated on seven JEE applications against four baselines, and separately validated with a survey of twenty-one industry practitioners, making it the reference point for what a “practical, effective” decomposition tool looks like. Its cost is instrumentation: a user must exercise the application’s use cases, or have functional tests do so, to obtain the traces the technique needs, and, being trace- and class-oriented, it targets Java applications exclusively. Underlying the graph-based branch of decomposition research more generally is the Louvain method of Blondel et al. [8], a greedy modularity-optimisation algorithm for community detection in large networks that, although developed outside the software-engineering literature for social and biological network analysis, has become a standard building block for graph-based module clustering in migration tools, including ALMS’s own Architect stage (Chapter 3).

2.2.3 LLM-Based Decomposition and Migration

Sellami and Saied’s MonoEmbed [4] replaces the hand-designed structural features used by earlier static approaches with language-model embeddings of source code, benchmarking eighteen pre-trained models and then improving the strongest of them with contrastive triplet fine-tuning before clustering with affinity propagation. Evaluated on eight Java monoliths, it achieves the best aggregate decomposition-quality score against six baselines including Mono2Micro, but its fine-tuning step requires a curated corpus of existing microservice repositories and GPU time that a single-laptop setting cannot assume; like Mono2Micro, it stops at the class-to-service partition and does not generate code. Maharjan et al. [9] take a related but distinct approach, applying a variational autoencoder-based graph neural network to the same decomposition problem and reporting a detailed case study of the resulting service boundaries, again without extending to code generation or deployment. Zhong et al. [10] instead apply spectral clustering directly to a dependency graph extracted from industrial legacy systems, an approach conceptually close to ALMS’s own graph-clustering boundary stage but evaluated only at the boundary-proposal level. Wang, Bornais, and Rubin [11] contribute an independent empirical comparison of several published microservice-decomposition tools under a common evaluation harness, and find that decomposition quality varies substantially by tool and input codebase - underscoring that no single decomposition technique, deterministic or learned, dominates across all inputs, which motivates ALMS’s decision to keep its own boundary-selection method simple, reproducible, and inspectable rather than pursuing marginal decomposition-quality gains.

Chen et al.’s Mono2SIs [5] is the closest prior work to ALMS in overall shape, and the system this survey positions ALMS against most directly. It pairs a lightweight static analysis, comparable in spirit to ALMS’s own Analyzer stage, with four sequential tool-using LLM agents that carry a Python or JavaScript monolith all the way to a deployed AWS serverless application. Evaluated on six benchmark applications, it reaches 100% deployment success and out-performs both commercial coding assistants and a same-input LLM baseline, but its agents run on hosted, commercial-scale backbones, calling out to a cloud API at every generation step. Its own ablation is the single most load-bearing result for ALMS’s design bet: removing the static-analysis-driven planning agent and falling back to an unguided agent drops deployment success from six of six to three of six, leading its authors to conclude that pipeline structure contributes more to migration quality than the strength of the backbone model - the finding that licenses ALMS’s central wager that a disciplined pipeline structure should let a small, local model substitute for a much larger hosted one. Kasdallah et al.’s MLStractor [12], published in 2026, extends this line of work by combining an LLM with search-based

optimisation for monolith-to-microservice decomposition specifically, reinforcing the trend toward LLM-assisted rather than purely learned or purely deterministic boundary selection, though again without reporting a deployed, running artifact. Yeh, Ma, and Chen [13] apply LLMs to a downstream migration artifact rather than to boundary selection: once service boundaries are already known, they use an LLM to migrate existing test cases from the monolith to its microservice targets, treating test migration as a distinct problem assumed to follow decomposition. Zan et al. [14] address a related code-generation challenge - generating code correctly against a private, previously unseen library - with a retrieval-augmented approach in which an API-finding module surfaces relevant documentation before a code-generation model is prompted, a retrieve-then-generate pattern that ALMS’s own retrieval-augmented generation stage (Chapter 3) applies to local framework and security pattern documents.

2.2.4 Local and Private LLM Deployment

Huang et al. [6] study the harder problem of deploying a closed-source model on an organization’s own infrastructure without exposing it to model-extraction attacks, proposing that only the early transformer layers need to be secured to preserve model confidentiality while still gaining the privacy and offline benefits of on-premises deployment. Hou et al. [15], in a broad systematic literature review of over three hundred studies on large language models for software engineering, find decomposition and migration to be one of many software-engineering tasks where LLM adoption has accelerated rapidly in the last two years, while reproducibility and deployment evaluation remain the exception rather than the rule across the surveyed literature - a finding consistent with what the migration-specific reviews above report, and one that motivates this project’s emphasis on a reproducible artifact assessed with candor rather than a headline decomposition-quality number.

2.3 Research Gap and Problem Definition

Read together, the fifteen references surveyed above converge on a single, consistent gap. Every prior automated pipeline reviewed here either stops at the class-to-service partition - as Mono2Micro [7], MonoEmbed [4], the variational-autoencoder approach [9], and the spectral-clustering approach [10] all do - or reaches a deployable artifact only by depending on hosted, commercial-scale language models and cloud inference, as Mono2SIs [5] does. No reviewed system combines a full decompose-generate-test-deploy pipeline with a fully local, zero-cost, offline execution model, and none of the systematic reviews of the area [1, 2, 3] identify one that does. This is precisely the gap that motivates the problem statement given in Chapter 1: whether a disciplined pipeline architecture can let a local, open-weight model occupy the point in this design

space that prior work has left unaddressed, at the explicit and acknowledged cost of the decomposition quality that fine-tuned or trace-based methods can achieve. Chapter 3 describes the methodology ALMS uses to attempt this, and Chapter 4 reports how far it gets.

No system surveyed across these fifteen references combines a complete migration pipeline with a fully local, offline, zero-cost execution model. Chapter 3 describes the methodology ALMS uses to address that gap.

Chapter 3

Methodology and Implementation

This chapter describes how ALMS is designed and built. Chapter 2 identified a specific gap in the literature: no reviewed system combines a complete decompose-generate-test-deploy pipeline with a fully local, offline, zero-cost execution model. This chapter presents the methodology ALMS uses to occupy that gap. Section 3.1 gives the overall block diagram of the system and the principle that organises it. Section 3.2 describes the hardware and software environment the system targets and was evaluated on. Section 3.3 describes each pipeline stage in detail, together with the algorithm each stage follows and the flowchart that governs the compile-retry cycle at the heart of code generation.

3.1 Block Diagram

ALMS takes a directory of Python source files as input and produces a set of containerised FastAPI services, one generated test suite per service, and a `docker-compose.yml` file that runs them together. The transformation is organised as a seven-stage pipeline (Fig. 3.1): (1) deterministic structural analysis, (2) service-boundary identification, (3) a human review gate, (4) per-service code generation inside a compile-retry loop, (5) per-service test generation, (6) a second human review gate, and (7) artifact serialisation. Stages 1–2 use no language model; stages 4–5 use a single local model; a retrieval component supplies pattern guidance to every model-facing stage.

The design is organised around one governing principle: **keep everything that can be made deterministic outside the language model, and give the model only the transformation step it is good at**. Boundary selection is fundamentally a graph-clustering problem, so ALMS solves it with community detection rather than by prompting a model. Output validation is fundamentally a compiler problem, so ALMS gates every generated service with Python’s `py_compile` and loops on failure. Inter-stage data contracts are schema-validation problems, so ALMS encodes them as typed records checked without a model in the loop - an LLM is never asked to judge another LLM’s output [5]. What remains for the model is: given a bounded set of source modules, a target framework, and retrieved idioms, emit the corresponding service. This division of labour is what allows a local model to stand in for the hosted commercial assistants used by comparable pipelines [4, 5]; prior work reports that pipeline structure

contributes more to migration quality than backbone-model strength [5].

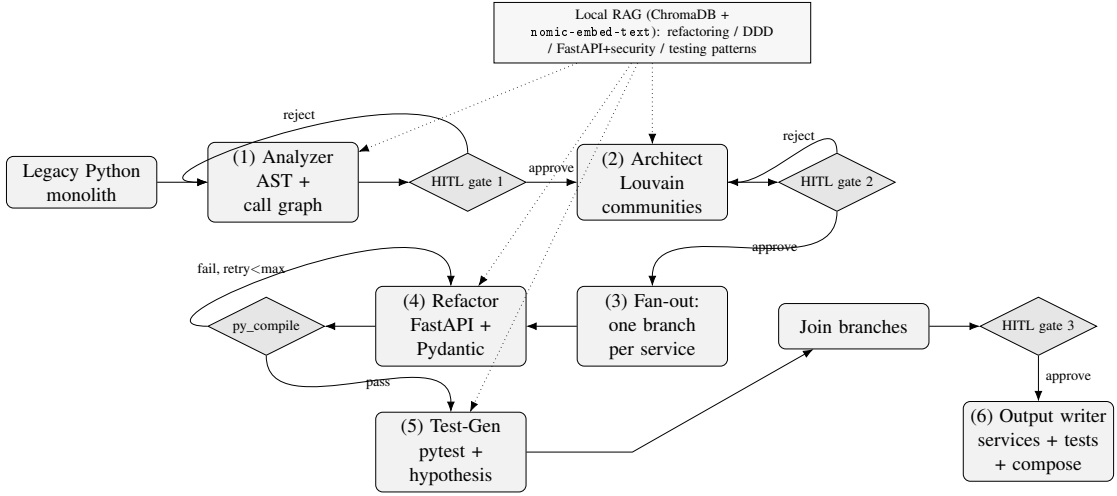


Figure 3.1: End-to-end ALMS migration pipeline: deterministic structural analysis and boundary identification feed a per-service fan-out in which a local LLM agent generates code (gated by a bounded compile-retry loop) and tests, checkpointed by three human-in-the-loop gates. All LLM stages draw on a local retrieval-augmented pattern knowledge base.

3.2 Hardware and Software Environment

Because ALMS is a software system rather than an embedded or physical product, its “hardware description” is the compute environment it targets and was evaluated on, rather than a physical component list (a full listing of software components and versions is given in Appendix III, and the environment specification is repeated in full in Appendix II). ALMS is designed to run entirely on a single commodity laptop with no dedicated GPU. The reference machine used for the reported evaluation (Chapter 4) is a CPU-only, 16 GB-RAM laptop with no discrete graphics processor. All language-model inference is served locally by Ollama on this same machine; no request is ever sent to a remote server, and no cloud compute or storage is provisioned.

The software environment is summarised in Table 3.1. Orchestration is implemented with LangGraph [16], a graph-based agent-orchestration library built on LangChain Core, which supplies the stateful graph, the cyclic compile-retry recovery loop described in Section 3.3.4, and the fan-out primitive used to dispatch one branch per proposed service. Language-model inference is served locally by Ollama [17]; the configuration evaluated in Chapter 4 uses `qwen3-coder:30b`, a Mixture-of-Experts model with 30.5 billion total parameters of which roughly 3.3 billion are active per token, quantised to approximately 19 GB on disk, together with `nomic-embed-text` for embeddings. A fully RAM-resident, lower-latency configuration using the smaller, fully dense `qwen2.5-coder:7b` model is also implemented and validated. Static analysis is built on Python’s own `ast` and `networkx` modules together with `python-louvain`

[18], an implementation of the Louvain community-detection method [8]. Retrieval-augmented generation is backed by ChromaDB, a local, SQLite-backed vector database queried by cosine similarity. Inter-agent data contracts are Pydantic v2 schemas. Generated code is post-processed with Jinja2 templates, black and isort formatting, and validated with py_compile; generated tests use pytest together with the property-based testing library hypothesis [19]. Target services are packaged as FastAPI applications backed by SQLAlchemy and Pydantic, containerised with Docker and docker compose.

Table 3.1: ALMS Software Environment and Technology Stack.

Concern	Choice
Orchestration	LangGraph $\geq 0.1.0$ [16] (StateGraph, Send fan-out, subgraph cycles), on LangChain Core
LLM runtime	Ollama [17], local; qwen3-coder:30b (MoE, 30.5B total / ~ 3.3 B active/token, Q4_K_M) - reported case study; qwen2.5-coder:7b (fully dense) also validated as a fully RAM-resident, lower-latency configuration
Embeddings	nomic-embed-text (via Ollama), 768-dimensional
Vector database	ChromaDB (local, SQLite-backed), cosine similarity
Static analysis	ast, networkx, python-louvain [18] (Louvain method [8])
Data contracts	Pydantic v2 (agent input/output schemas)
Codegen post-processing	Jinja2, black, isort, py_compile
Static security scan	Bandit (optional)
Generated tests	pytest, hypothesis [19]
Target services	FastAPI + SQLAlchemy + Pydantic, Docker / docker compose
Cache / audit	DiskCache, SQLite
Console UI	rich

3.3 Software Description, Flowchart and Algorithm

3.3.1 Deterministic structural analysis

The Analyzer parses every .py file with Python’s ast module and extracts, per module: function definitions (name, parameters, called names, cyclomatic complexity, lines of code, docstring), class definitions (methods, base classes), import/from...import statements, and module-level assignments. Cyclomatic complexity is computed by

counting decision points (`if`, `for`, `while`, `except`, boolean operators) directly, rather than invoking an external metric tool, so the metric is reproducible across environments.

From this structure the Analyzer builds a directed dependency graph whose nodes are modules, functions, and classes. Edges are added only for relationships that correspond to real code entities: a call is linked to its resolved internal target if one exists, or to a third-party module if the call’s head matches a known import; inheritance edges are added only to classes or imported modules observed in the source. Calls that resolve to Python builtins, standard-library methods, or local-variable methods are deliberately **not** added as edges, because they are not architectural dependencies and would otherwise dominate both the coupling analysis and the clustering step that follows (Chapter 4 quantifies this effect). The graph is then reduced to a list of coupling hotspots (modules whose cross-module fan-out exceeds a threshold) and a set of circular dependencies, found via strongly connected components. A SHA-256 hash over the sorted source contents is computed and used as a cache key, so re-running the pipeline on an unchanged codebase replays cached agent outputs instead of re-invoking the model. The Analyzer’s output is a typed record - codebase statistics, graph nodes and edges, hotspots, external dependencies, and cycles - validated by schema, with no language model involved in its construction. Static call-graph extraction for Python is also available as a standalone visualisation tool: `Pyan3` [20] performs offline AST-based call-graph generation comparable in spirit to the first half of this stage, but stops at producing a call graph for human inspection (typically rendered with `Graphviz`) rather than feeding it into community detection or any downstream boundary-selection or code-generation step, as the Analyzer does here.

3.3.2 Service boundary identification

The Architect converts the directed dependency graph into an undirected weighted graph, using edge confidence as the weight, and applies the Louvain community-detection algorithm [8] (via `python-louvain` [18]) with a fixed random seed. Each resulting community becomes a candidate service. The service name is derived deterministically from the community’s constituent modules, with a numeric suffix appended on collision so that distinct communities never produce colliding service identifiers. Each candidate is emitted as a boundary record: name, bounded-context description, member modules, proposed endpoints, a confidence score, and a short rationale.

Optionally, the Architect prompts the local model with the analysis summary and retrieved domain-driven-design guidance to refine or re-group the communities; when the model returns a well-formed set of boundaries these are used, otherwise the pipeline falls back to the raw community assignment. Because the fallback is deterministic and seed-fixed, the same input codebase yields the same decomposition on every run - a

property that hosted, non-deterministic pipelines cannot offer [5]. ALMS trades the decomposition quality of learned or trace-based methods [4, 7] for this determinism, and for requiring neither training data, a GPU, nor runtime instrumentation. Chapter 4 reports a concrete case in which this approach assigns a coherent bounded context to three of four communities but produces a fourth, low-confidence, non-domain “catch-all” service from the monolith’s residual orchestration glue code.

3.3.3 Retrieval-augmented generation

Every model-facing stage is grounded in a local knowledge base of short, curated migration-pattern documents. In the configuration evaluated in Chapter 4, the knowledge base held 16 documents across five categories: FastAPI (5), refactoring (4), Docker/deployment (4), domain-driven design (2), and security (1). Documents are split into 500-character chunks with 100-character overlap, embedded with a local embedding model, and stored in a persistent on-disk vector index using cosine similarity. Each agent is bound to the subset of categories relevant to its task - the Analyzer to refactoring patterns, the Architect to domain-driven-design patterns, the Refactoring agent to FastAPI and security patterns, and the Test-Gen agent to a testing-patterns category that the deployed knowledge base does not currently populate (Chapter 5 returns to this) - and retrieves up to a configured number of chunks per query, filtered by a 0.70 cosine-similarity threshold. This retrieve-then-generate pattern mirrors approaches used elsewhere for grounding code generation in retrieved API documentation [14, 5], and substitutes for framework knowledge that a small local model does not reliably possess on its own.

3.3.4 Code generation and the compile-retry loop

For each proposed service, the Refactoring agent receives the service’s member modules with full function bodies (unlike the Analyzer, which sees only AST-level summaries), the target framework, and retrieved FastAPI and security patterns, and emits one or more Python files implementing the service as a FastAPI application with Pydantic schemas and SQLAlchemy models. Generated code is formatted with `black` and `isort` and then passed through a `py_compile` gate. If compilation fails, the orchestrator loops back to the Refactoring agent with the error message, for up to three attempts (configurable). If the service still does not compile after the last attempt, it is marked as needing human review and carried forward with that status rather than dropped. Figure 3.2 shows this cycle as it executes inside each per-service branch.

A representative excerpt of generated output, from the User Management Service produced for the case study reported in Chapter 4, is shown below: a FastAPI endpoint generated directly from the legacy monolith’s `users.create_user` function, translated into the framework’s dependency-injected, schema-validated idiom.

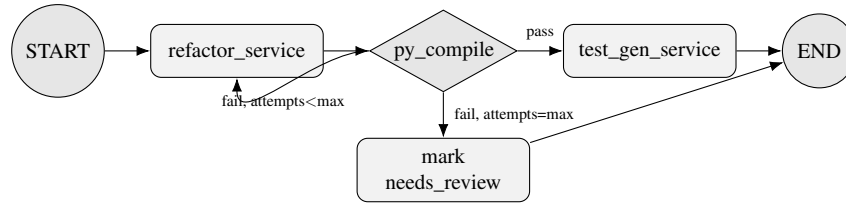


Figure 3.2: LangGraph per-service subgraph (`process_service`), run once per fan-out branch. Test generation is reached only on a compile pass and has no equivalent retry cycle of its own (Section 3.3.5, Section 4.2).

```

@app.post("/users", response_model=User)
async def create_user(
    user: UserCreate,
    db: Session = Depends(get_db)
):
    existing = db.query(UserORM)\
        .filter(UserORM.email ==
                user.email).first()
    if existing:
        raise HTTPException(
            status_code=400,
            detail=f"User with email "
                f"{user.email} exists")
    hashed = hash_password(user.password)
    db_user = UserORM(
        email=user.email,
        name=user.name,
        hashed_password=hashed,
        role=user.role,
        is_active=user.is_active)
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user
  
```

This function compiles against a Pydantic `UserCreate` schema with a field validator enforcing a minimum password length, and a SQLAlchemy `UserORM` model with a uniqueness constraint on email - a direct, compiling translation of the legacy validation logic rather than a paraphrase of it.

3.3.5 Test generation and shadow testing

For each service that compiles, the Test-Gen agent generates a pytest suite - happy-path, error-path, and property-based (hypothesis [19]) cases - from the generated service code and the corresponding legacy source. ALMS additionally includes a shadow-testing engine that runs identical inputs through a legacy callable and its generated counterpart and compares outputs for an exact match, targeting behavioural parity rather than

only syntactic validity. Unlike the Refactoring agent (Section 3.3.4), Test-Gen output is not currently passed through a `py_compile` gate or a retry loop: Figure 3.2 routes a compiled service directly to test generation with no cyclic recovery path. Chapter 4 reports the measurable consequence of this asymmetry.

3.3.6 Orchestration: fan-out, reducers, and human gates

The pipeline as a whole is a stateful graph. After boundary approval, the orchestrator issues one parallel branch per proposed service; each branch runs the compile-retry loop and then test generation as an isolated subgraph with its own attempt counter (Figure 3.2). Branch results are merged back into a single list by a custom reducer keyed on service name, so that parallel branches never race and a re-run after a rejected final gate replaces a service’s result rather than duplicating it.

Three human-in-the-loop gates punctuate the pipeline: after analysis, after boundary proposal, and after test generation (Figure 3.1). At each gate a reviewer may approve (continue), reject with feedback (re-run the preceding stage with that feedback), or stop (terminate). A command-line flag `auto-approves` every gate for unattended runs. These gates make the decomposition and the generated output auditable checkpoints rather than opaque end-to-end steps.

3.3.7 Output and data contracts

On completion, the orchestrator writes, per service, the generated Python module(s), a `requirements.txt`, and a Dockerfile; a top-level `docker-compose.yml` wires the services on distinct ports behind an Nginx API gateway; and the generated test suites plus the analysis and architecture records are serialised as JSON. Every inter-stage message is a Pydantic v2 model, validated on construction, so a malformed agent response is caught at the boundary rather than propagating downstream, and no model is used to validate another model’s output. The system is invoked from a single command-line entry point: `-init-kb` builds the knowledge base, `-demo` runs the pipeline on a bundled sample monolith, `-skip-hitl` runs unattended, and `-check` verifies the local environment. Reproducibility rests on three choices: a fixed Louvain seed makes boundary selection deterministic, a codebase hash makes cached runs replay exactly, and local open-weight models remove dependence on a hosted endpoint whose behaviour can drift between runs.

ALMS’s seven-stage pipeline keeps boundary selection, output validation, and inter-agent communication deterministic, and reserves the local language model for the code- and test-generation steps it is suited to, all on a single CPU-only laptop with the software stack summarised in Table 3.1.

Chapter 4

Results and Analysis

Chapter 3 described how ALMS is designed to keep boundary selection, output validation, and inter-agent communication deterministic while reserving a local language model for code and test generation. This chapter reports what happened when we exercised that design end to end on a representative case-study codebase. The evaluation is deliberately framed as a feasibility study rather than a benchmark: it reports what a single, fully local pipeline run produces, including where that run fell short, rather than a comparison across many codebases. Section 4.1 describes the evaluation methodology and the standards it draws on. Section 4.2 reports results organised around four research questions. Section 4.3 walks through one service end to end as a worked example.

4.1 Evaluation Methodology and Standards Adherence

ALMS is evaluated on `examples/sample_monolith`, a 6-file, 1,578-line Flask application (76 functions, 7 classes, mean cyclomatic complexity 2.32) spread across six modules - `app`, `database`, `models`, `orders`, `payments`, and `users`. The full pipeline is run on this input on the CPU-only, 16 GB-RAM reference laptop described in Section 3.2, and its output is checked at four levels: whether generated code compiles, whether generated containers boot behind a working API gateway, whether the deterministic graph-filtering step reduces the boundary-selection graph as designed, and what resource and cost profile the run has relative to a hosted-LLM pipeline. We do not claim that these results generalise beyond this one codebase; Chapter 5 treats that limitation explicitly as part of the system’s limitations.

Software test documentation and reporting in this evaluation is organised along the lines recommended by IEEE/ISO/IEC verification-and-validation practice for software artifacts, in particular the separation of test case generation, execution, and result reporting that IEEE 829-style test documentation and the software verification and validation guidance in IEEE 1012 recommend¹; no formal certification or compliance audit against either standard was carried out, and this report does not claim one. The generated test suites themselves follow standard `pytest` conventions together with property-based testing via `hypothesis` [19], and container validity is checked by the same build-and-boot criterion an operations team would apply before promoting a service.

¹IEEE Standards Association, <https://www.ieee.org/content/ieee-org/en/standards/index.html/>.

4.2 Results

4.2.1 RQ1: Does the pipeline complete on commodity, CPU-only hardware?

We ran the pipeline with `python main.py` on the CPU-only, 16 GB-RAM reference laptop, with all inference served locally by Ollama (`qwen3-coder:30b` for generation, `nomic-embed-text` for embeddings) and no GPU involved. The pipeline reached its terminal (`complete`) phase in a single unattended run whose total wall-clock exceeded ten hours - consistent with the CPU-only, memory-mapped weight-paging behaviour described in Section 3.2, since the model’s approximately 19 GB of quantised weights do not fit in the host’s 16 GB of physical RAM. Because Ollama serves the model locally and the disk cache replays agent outputs for an unchanged codebase hash, no run of this pipeline made a network call or incurred an API charge. Per-stage timing and peak-RAM figures were not retained alongside the generated artifact for this run; we report this as a limitation in Chapter 5 rather than present invented numbers. A companion run of the fully RAM-resident `qwen2.5-coder:7b` configuration completed the equivalent pipeline in substantially less wall-clock time on the same hardware, at the cost of the lower generation quality reflected in that run’s smaller, three-service output; a controlled, timed comparison between the two configurations is left as future work (Chapter 6).

4.2.2 RQ2: Is the generated output syntactically valid and bootable?

Code generation. The Architect proposed 4 service boundaries; all 4 were carried through code generation, and all 4 of the generated service modules pass `py_compile` with no service requiring the fallback “needs human review” status - consistent with the compile-retry loop design of Section 3.3.4.

Test generation. The Test-Gen agent produced 570 test files (1,120 test cases by the pipeline’s own count) across the four services. Because Test-Gen output is not passed through a compile gate (Section 3.3.5), we checked this directly: only 332 of the 570 generated test files (58.2%) are importable, syntactically valid Python. The remaining 238 files (41.8%) share a single, consistent failure mode - an unterminated `from app import` (statement partway through the file’s fixture header, together with decorator-only fixture stubs with no following function body - consistent with output truncation against the Test-Gen agent’s context window rather than a logic error. This is a direct, measurable consequence of the asymmetry noted in Section 3.3.5: the same discipline that gives code generation a 100% `py_compile` pass rate is not currently applied to test generation.

Container deployment. We ran a full `docker compose up` of the generated stack, including the Nginx API gateway, against all four services. Three of the four containers

- User Management Service, Order Processing Service, and Payment Processing Service - built and started successfully behind a working gateway. The fourth, Application Core Service, did not start. Application Core Service is notable in the Architect’s own output: it is the lowest-confidence proposal (0.78 against 0.87–0.92 for the other three), owns no database tables, and is described by the Architect itself as a service that “acts as an API gateway and orchestrator, coordinating calls between other services” - that is, it is the monolith’s residual request-handling glue code that the clustering step could not assign a clean bounded context, rather than a genuine domain service. Chapter 5 returns to this as a limitation. We also observed that the Order Processing and Payment Processing services independently regenerate CRUD endpoints and models for `/users` and `/products` that are also, correctly, owned by User Management Service - a data-ownership leak that the current schema contracts do not detect or prevent.

4.2.3 RQ3: What effect does deterministic structural filtering have?

Unlike RQ1 and RQ2, this result requires no language model and is fully reproducible from the static analysis alone: the dependency graph the Analyzer would build without the builtin/stdlib/local-variable call filter described in Section 3.3.1 is compared against the graph it builds, both computed directly from the current implementation against `sample_monolith`. The unfiltered graph has 171 nodes and 267 edges and yields 6 coupling hotspots; the filtered graph used by ALMS has 75 nodes and 104 edges and yields 2 hotspots - a 56.1% reduction in nodes and a 61.0% reduction in edges (Figure 4.1). The two hotspots that survive filtering are `app`, which fans out to `database`, `orders`, `payments`, and `users`, and `orders`, which fans out to `database`, `models`, `payments`, and `users` - the two genuine god-modules in this monolith, as opposed to the mixture of god-modules and spurious builtin/stdlib “coupling” that the unfiltered graph reports. This matters beyond hotspot reporting: the same filtered graph is the input to Louvain community detection (Section 3.3.2). An unfiltered graph in which every function is nominally coupled to `len`, `round`, and `ValueError` pollutes the clustering with edges that carry no architectural meaning.

4.2.4 RQ4: What is the resource and cost profile relative to a hosted pipeline?

ALMS is not compared against Mono2SIs [5] on decomposition or correctness - the two systems target different output platforms (portable containers versus AWS SAM) and were not run on the same benchmark. The comparison that is meaningful, reported in Table 4.2, is qualitative: Mono2SIs’s agents run on hosted, commercial-scale backbones and its own reported per-agent latency is 58–106 minutes, dominated by paid cloud inference [5]; ALMS’s entire run, including analysis, boundary selection, generation, and testing, costs \$0 in API fees and never leaves the host machine, at the explicit

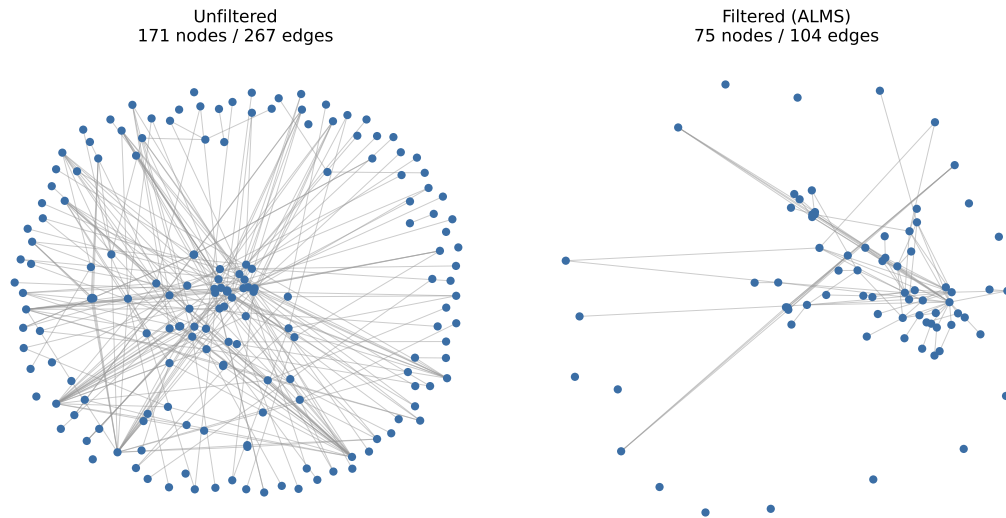


Figure 4.1: Dependency graph of `examples/sample_monolith` before and after deterministic filtering of non-architectural calls (Python builtins, `stdlib` methods, local-variable methods), recomputed directly from the current Analyzer implementation. Filtering reduces the graph from 171 nodes / 267 edges / 6 hotspots to 75 nodes / 104 edges / 2 hotspots (56.1% / 61.0% reduction).

cost of the multi-hour wall-clock documented in RQ1 for a model large enough to be paged from disk. The trade-off is explicit and one-directional: ALMS does not claim to match Mono2SIs’s deployment success or its end-to-end correctness benchmark; it claims to reach a smaller but non-trivial point in the same design space without a cloud dependency, a GPU, or a per-run bill.

Table 4.1: Resource and Cost Profile (`sample_monolith`, CPU-only).

Item	Value
Host	CPU-only laptop, 16 GB RAM, no GPU
Model (reported run)	qwen3-coder:30b (MoE, ~3.3B active/token)
Model weights on disk	~19 GB (Q4_K_M), exceeds host RAM
Serving strategy	Ollama, CPU memory-mapped weight paging
Embedding model	nomic-embed-text
Total wall-clock	>10 h (single unattended run)
Per-stage timing	Not instrumented for this run (Chapter 5)
Peak RAM	Not instrumented for this run (Chapter 5)
API cost	\$0
Network	Offline throughout

Table 4.2: Comparison of Monolith-to-Microservice Migration Systems. “Reach” = Decompose → Generate → Test → Deploy.

System	Boundary method	Reach	LLM(s) / hardware / cost
Mono2Micro [7]	Hierarchical spatio-temporal trace clustering	Decompose only	None; local, no LLM
MonoEmbed [4]	LLM embeddings + contrastive fine-tuning → affinity propagation	Decompose only	18 benchmarked LLMs; GPU for fine-tuning
Mono2SIs [5]	LLM Architect agent over static call/entry-point map	Full, to AWS SAM	Hosted commercial-scale models; cloud inference; \$0.58–1.06/min-app
Yeh et al. [13]	- (boundaries assumed)	Test only	Hosted LLM (test migration only)
ALMS (this work)	Deterministic AST call graph → filtering → seed-fixed Louvain; LLM refines	Full, local containers	qwen3-coder:30b / qwen2.5-coder:7b, local; single CPU laptop, 16 GB RAM, no GPU; \$0

4.3 Case Study Walkthrough

Given `sample_monolith`’s six modules, the Analyzer’s filtered dependency graph (Section 4.2.3) is passed to the Architect, which runs Louvain community detection with a fixed seed. In the reported run, the LLM-refinement step returned a well-formed boundary set for three of the four proposed services - **User Management Service** (users; tables users, user_profiles, sessions, roles; confidence 0.92), **Order Processing Service** (orders; tables orders, order_items, products, order_status_transitions; confidence 0.89), and **Payment Processing Service** (payments; tables payments, refunds, payment_gateways, payment_transactions; confidence 0.87) - plus the lower-confidence **Application Core Service** discussed in Section 4.2.2 (confidence 0.78, no owned tables).

For User Management Service, the Refactoring agent produced a 359-line FastAPI module exposing 13 endpoints, including the `create_user` endpoint listed in Section 3.3.4, against a Pydantic `UserCreate` schema with a field validator enforcing a minimum password length, and a SQLAlchemy `UserORM` model with a uniqueness constraint on email - a direct, compiling translation of the legacy `users.create_user` function’s validation logic into FastAPI’s dependency-injected, schema-validated idiom. The corresponding generated test file - one of the 332 syntactically valid test files (Section 4.2.2) - exercises this endpoint against a test client and asserts on the response body’s email, name, role, default-active flag, and generated identifier and creation-timestamp fields. The complete stack is wired by the generated `docker-compose.yml`, exposing each service on a distinct port behind the Nginx gateway on port 8080, which

is the artifact this case study evaluates for build and boot validity in Section 4.2.2.

On the Flask monolith used for this case study, the pipeline ran end to end on CPU-only hardware at zero API cost. All four generated services compiled cleanly, though only 58.2% of generated test files were syntactically valid. Deterministic structural filtering cut the boundary-selection graph by more than half before any model saw it, and three of the four generated services booted behind a working API gateway. Chapter 5 weighs what these results say about the system’s advantages, limitations, and applications.

Chapter 5

Advantages, Limitations and Applications

Chapter 4 reported what ALMS produced when run end to end on a representative case-study codebase: a mostly-bootable, containerised microservice stack generated entirely on a single CPU-only laptop. This chapter steps back from that single run to assess the system more broadly - what its design buys a user (Section 5.1), where the current implementation falls short and why (Section 5.2), and who is realistically positioned to benefit from a system built this way (Section 5.3).

5.1 Advantages

The central advantage of ALMS’s design is captured by four measurements from the case study in Chapter 4, each of which follows directly from the decision to keep the language model out of every step that does not strictly need it (Section 3.1). The evaluated run cost \$0 in API fees, since every inference call is served locally by Ollama rather than a hosted provider. It ran 100% offline: no stage of the pipeline, from static analysis through container packaging, made a network call at any point. It required zero GPUs, running entirely on a CPU-only, 16 GB-RAM laptop, including for a 30-billion-parameter Mixture-of-Experts model served through memory-mapped weight paging. And zero lines of the source codebase ever left the machine performing the migration, since no code is transmitted to a third-party API at any stage.

Prior automated pipelines, as Chapter 2’s literature survey established, tend to pick one of two positions: reach a deployable result by depending on hosted, commercial-scale models and cloud inference [5], or stay local and stop at a class-to-service partition without ever producing a running artifact [7, 4]. ALMS combines a full decompose-generate-test-deploy pipeline with a fully local, offline execution model - a combination the systematic reviews of this area do not report any prior system offering [1, 2, 3]. This matters specifically for the constituency identified in Chapter 1: students and independent developers without a GPU or cloud budget, small teams for whom per-run API fees are a real constraint, and organisations that cannot send proprietary source code to a third-party service for privacy or regulatory reasons. For that constituency, a slower, less polished result that runs entirely on hardware they already own is a meaningfully different proposition from a faster result that requires infrastructure they do not have.

A second advantage, orthogonal to cost, is auditability. Because boundary selection

is a seed-fixed deterministic algorithm rather than a model call, the same input codebase produces the same decomposition on every run (Section 3.3.2) - a property that hosted, non-deterministic pipelines cannot offer [5]. Because every generated service is gated by `py_compile` with bounded automatic retry (Section 3.3.4), the reported case study reached a 100% compile pass rate with no manual intervention. And because three human-in-the-loop checkpoints punctuate the pipeline (Section 3.3.6), a reviewer can inspect and reject the analysis, the proposed boundaries, or the generated tests before they are carried forward, rather than receiving only a final, opaque result.

5.2 Limitations

The case study in Chapter 4 also surfaced three concrete limitations, each of which is a direct consequence of a specific design choice rather than an incidental bug, and each of which motivates a specific item in Chapter 6’s future work.

The Application Core Service failure. Louvain community detection assigns every module to some community; when a module’s role in the codebase is orchestration and request routing rather than domain ownership - as the monolith’s `app.py` entry point is in the evaluated case study - the clustering step has no mechanism to recognise that, and still emits a community for it, which the Architect then describes with a plausible-sounding bounded-context rationale. The resulting service has no data to own, no clear single responsibility, and, in the reported run, does not start. A pipeline that wanted to handle this case would need either a pre-clustering heuristic that flags orchestration-only hub modules for different treatment, or a post-clustering check that routes a proposed service without owned tables to mandatory human review rather than auto-approval; neither exists in the current implementation.

The test-generation compile-validity gap. Only 58.2% of the 570 generated test files are syntactically valid Python, against a 100% `py_compile` pass rate for generated service code (Section 4.2.2). This is not a separate, unrelated defect; it is the same underlying lesson the rest of this report argues for, observed in the one place the current implementation has not yet applied it. The stages of ALMS that are protected by a deterministic gate are reliable, and the one model-facing stage that lacks an equivalent gate is measurably less so.

Cross-service data duplication. The Order Processing and Payment Processing services independently regenerate CRUD endpoints and models for `/users` and `/products` that are also, correctly, owned by User Management Service (Section 4.2.2). Data ownership as declared by the Architect is not currently enforced when sibling services are later generated, so nothing in the current schema contracts catches a service that quietly duplicates another service’s owned resource.

Beyond these three case-study-specific findings, the evaluation itself has a scope limitation that Chapter 4 was explicit about: it is a single case study on one 1.6-kLOC Flask monolith, run once with one model configuration. No result in this report should be read as evidence about performance on larger codebases, other Python web frameworks, non-web applications, or languages other than Python, and generation quality depends on language-model sampling at non-zero temperature, so run-to-run variance in retry counts and generated-code content is uncharacterised beyond the single reported run.

5.3 Applications

The design trade-offs described above point toward a specific set of users and settings rather than toward general-purpose migration tooling. Students and academic researchers working on software-modernisation coursework or research projects can use ALMS to obtain a working, if imperfect, decomposition of a small Python codebase without needing institutional cloud credits, a GPU-equipped workstation, or permission to send assignment or research code to a third-party API - precisely the setting this project itself was developed and evaluated in. Independent developers and small teams evaluating whether a legacy Python service is a good candidate for microservice decomposition can use ALMS as a low-cost, zero-commitment first pass: a proposed boundary set, a compiling code skeleton, and a bootable container stack that a team can inspect and revise, rather than a production-ready migration.

Organisations in privacy-regulated or security-sensitive industries - healthcare, finance, government, and defence contractors, among others - where sending proprietary source code to a hosted large-language-model API is restricted or prohibited by policy, form a third natural audience: ALMS’s entirely local execution model is a structural property, not a configuration option, so no code generated or analysed by the pipeline ever leaves the machine it runs on. Finally, resource-constrained teams and organisations more generally - those without a dedicated GPU budget or the willingness to accrue a per-run API bill for an exploratory migration - can use ALMS to obtain a first migration attempt at zero marginal cost beyond the hardware they already own, accepting the multi-hour wall-clock and the case-study-scale limitations documented in this report as the price of that access.

ALMS’s zero-cost, fully local, auditable design serves students, small teams, and privacy-constrained organisations that prior hosted-LLM pipelines exclude by construction. Its three limitations each trace back to a specific place the deterministic/LLM design principle has not yet been applied, and its realistic applications follow directly from the constituency its offline design is built to serve. Chapter 6 concludes the report

and outlines the future work that follows from these limitations.

Chapter 6

Conclusion and Future Scope

ALMS migrates a Python monolith to containerised FastAPI microservices on a single commodity laptop, using a local language model, at zero API cost, offline, and reproducibly. Its central design decision, introduced in Chapter 3, is a strict separation of concerns: boundary selection, output validation, and inter-agent communication are handled deterministically - by a filtered call graph, a seed-fixed community-detection algorithm, a `py_compile` gate with bounded regeneration, and schema-checked data contracts - so that the language model is responsible only for the code- and test-generation steps it is suited to. The case-study evaluation in Chapter 4 carried one representative monolith through the full pipeline to a mostly-bootable container stack, three of four services running behind a working API gateway, using a 30-billion-parameter Mixture-of-Experts local model an order of magnitude larger than prior local-only feasibility work has reported; deterministic structural filtering cut graph nodes by 56.1% and edges by 61.0% before boundary selection ran. Chapter 5 reported what did not work with the same directness: one non-domain service that community detection could not cleanly assign a bounded context to, a test-generation stage whose output validity (58.2%) lags the code-generation stage's (100%) because it lacks the same compile-retry gate, and cross-service data-ownership leakage that the current schema contracts do not catch.

ALMS does not match, and does not claim to match, the decomposition quality of fine-tuned or trace-based prior work [4, 7], or the deployment scale of pipelines built on hosted commercial models [5]. Its contribution, as Chapter 2's literature survey established and Chapter 5 elaborated, is access: evidence that a full migration pipeline, including a substantially larger local model than prior local-only baselines have attempted, can run on hardware a single developer already owns, with an honest account of where that pipeline currently falls short.

The most direct extension of this work is scale: running ALMS unattended over a corpus of 10–30 small and medium Python monoliths would replace the single case study in Chapter 4 with per-repository and aggregate statistics, and would let ablations of the design - no retrieval, no compile-retry loop, no deterministic clustering, no human-in-the-loop gates - be measured rather than argued for. Closing the behavioural-fidelity gap identified in Chapter 5 requires wiring the existing shadow-testing engine (Section 3.3.5) to a live `docker compose up` deployment and replaying recorded monolith traffic against it, rather than comparing in-process callables. On the decom-

position side, deriving table ownership and inter-service call records from the static analysis itself - rather than leaving them populated only when the model chooses to - and allowing a reviewer to interactively nudge the Louvain community assignment at the human-in-the-loop gate would address the hub-module over-merging problem identified in Section 5.2 directly, rather than only reporting it; the same mechanism could flag a proposed service with no owned tables for mandatory review instead of auto-approval.

Extending the same `py_compile`-gated, bounded-retry discipline that Section 3.3.4 gives code generation to the Test-Gen agent’s output (Section 4.2.2, Section 5.2) is a small, concrete, and directly motivated next step, as is enforcing the data-ownership boundaries the Architect already declares so that sibling services stop regenerating one another’s CRUD surface. Populating the currently empty testing-patterns knowledge-base category (Section 3.3.3) and instrumenting per-stage timing and peak-RAM consistently across runs (Section 4.2.4) would close two further gaps this report’s resource table reports as absent rather than measured. Finally, a controlled, timed comparison of the fully RAM-resident `qwen2.5-coder:7b` configuration against the paged `qwen3-coder:30b` configuration reported here - decomposition quality, compile and boot success rate, and wall-clock, on the same input - would turn Section 4.2.1’s informal observation into a quantified result. The AST-based front end is Python-specific by construction; supporting additional languages would require a parallel structural analyzer per language, sharing the downstream boundary-selection, generation, and validation stages described in Chapter 3.

References

- [1] H. Hassan, M. A. Abdel-Fattah, and W. Mohamed, “Migrating from monolithic to microservice architectures: A systematic literature review,” *International Journal of Advanced Computer Science and Applications*, vol. 15, no. 10, pp. 104–116, Oct. 2024.
- [2] A. Martínez Saucedo, G. Rodríguez, F. Gomes Rocha, and R. Pereira dos Santos, “Migration of monolithic systems to microservices: A systematic mapping study,” *Information and Software Technology*, vol. 177, art. 107590, 2025.
- [3] Y. Abgaz, A. McCarren, P. Elger, D. Solan, N. Lapuz, M. Bivol, G. Jackson, M. Yilmaz, J. Buckley, and P. Clarke, “Decomposition of monolith applications into microservices architectures: A systematic review,” *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4213–4242, 2023.
- [4] K. Sellami and M. A. Saied, “MonoEmbed: Enhancing LLM representations for monolith to microservices decomposition through contrastive learning,” *Empirical Software Engineering*, vol. 31, no. 1, art. 11, 2026.
- [5] X. Chen, Y. Su, Z. Su, Y. Yu, and Z. Zheng, “Mono2SIs: Automated monolith-to-serverless migration via multi-stage pipeline with static analysis,” *arXiv preprint arXiv:2604.24550*, 2026.
- [6] H. Huang, Y. Li, B. Jiang, B. Jiang, L. Liu, R. Sun, Z. Liu, and S. Liang, “A middle path for on-premises LLM deployment: Preserving privacy without sacrificing model confidentiality,” *arXiv preprint arXiv:2410.11182*, Oct. 2024.
- [7] A. K. Kalia, J. Xiao, R. Krishna, S. Sinha, M. Vukovic, and D. Banerjee, “Mono2Micro: A practical and effective tool for decomposing monolithic Java applications to microservices,” in *Proc. 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2021, pp. 1214–1224.
- [8] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, “Fast unfolding of communities in large networks,” *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, art. P10008, Oct. 2008.

- [9] R. Maharjan, K. Sooksatra, T. Cerny, Y. Rajbhandari, and S. Shrestha, “A case study on monolith to microservices decomposition with variational autoencoder-based graph neural network,” *Future Internet*, vol. 17, no. 7, art. 303, Jul. 2025.
- [10] T. Zhong, Y. Teng, S. Ma, J. Chen, and S. Yu, “A microservices identification method based on spectral clustering for industrial legacy systems,” *arXiv preprint arXiv:2312.12819*, Dec. 2023.
- [11] Y. Wang, S. Bornais, and J. Rubin, “Microservice decomposition techniques: An independent tool comparison,” in *Proc. 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Sacramento, CA, USA, 2024.
- [12] I. Kasdallah, M. A. Ghorab, O. Jebbar, K. Sellami, M. Sayagh, A. Ouni, and M. A. Saied, “MLStractor: LLM-powered search-based monolith-to-microservice decomposition,” in *Search-Based Software Engineering (SSBSE 2026)*, Lecture Notes in Computer Science, vol. 16699, Springer, Cham, 2026, pp. 83–89.
- [13] H.-W. Yeh, S.-P. Ma, and Y. Chen, “Test case migration from monolith to microservices using large language models,” in *2024 IEEE International Conference on e-Business Engineering (ICEBE)*, 2024, pp. 29–35.
- [14] D. Zan, B. Chen, Y. Gong, J. Cao, F. Zhang, B. Wu, B. Guan, Y. Yin, and Y. Wang, “Private-library-oriented code generation with large language models,” *arXiv preprint arXiv:2307.15370*, Jul. 2023.
- [15] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, “Large language models for software engineering: A systematic literature review,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, art. 220, Dec. 2024.
- [16] LangChain AI, “LangGraph,” GitHub repository. [Online]. Available: <https://github.com/langchain-ai/langgraph>
- [17] Ollama Inc., “Ollama: Get up and running with large language models locally.” [Online]. Available: <https://ollama.com/>
- [18] T. Aynaud, “python-louvain: Louvain Community Detection,” GitHub repository. [Online]. Available: <https://github.com/taynaud/python-louvain>
- [19] D. R. MacIver, Z. Hatfield-Dodds, and the Hypothesis Contributors, “Hypothesis: A new approach to property-based testing,” *Journal of Open Source Software*, vol. 4, no. 43, art. 1891, 2019.

- [20] E. Horner and J. Jeronen, “Pyan3: Offline call graph generator for Python,” GitHub repository. [Online]. Available: <https://github.com/Technologicat/pyan3>

Appendix I

Soft Code Flowcharts

This appendix reproduces, for consolidated reference, the two pipeline flowcharts introduced in Chapter 3.

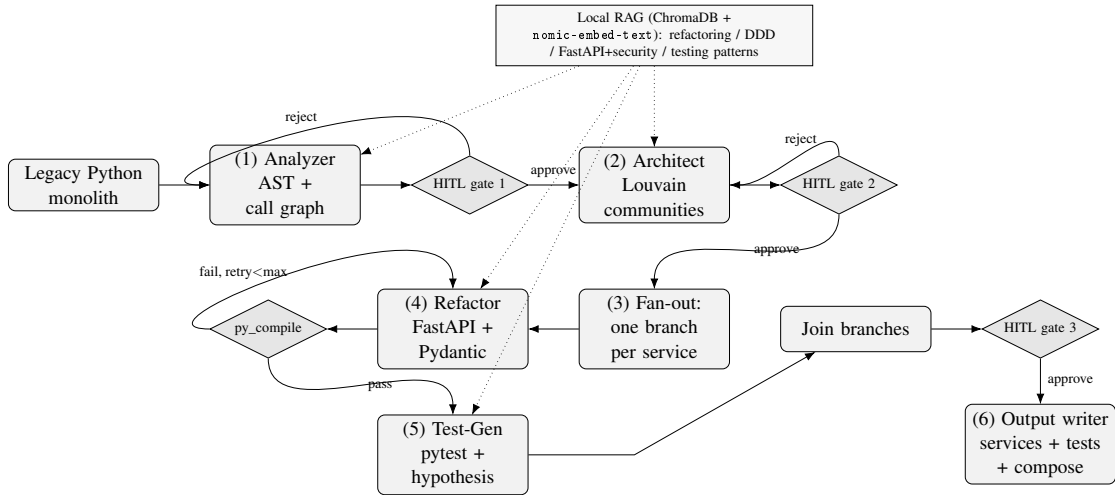


Figure I.1: Reproduction of Figure 3.1: end-to-end ALMS migration pipeline flowchart.

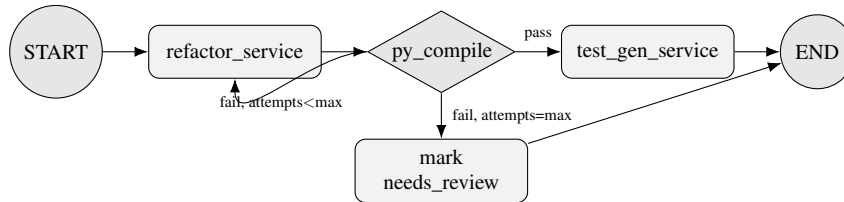


Figure I.2: Reproduction of Figure 3.2: LangGraph per-service compile-retry subgraph flowchart.

Appendix II

Software and Hardware Environment Specifications

This appendix details the environment used to develop and evaluate ALMS, expanding on the summary given in Section 3.2.

Host machine. CPU-only laptop, 16 GB physical RAM, no discrete or integrated GPU used for inference. All storage is local disk; no cloud compute, storage, or inference endpoint is provisioned at any point.

Model runtime. Ollama, running locally, serving all inference requests over a local API with no external network call. Two model configurations are implemented and validated:

- `qwen3-coder:30b` - Mixture-of-Experts architecture, 30.5 billion total parameters, approximately 3.3 billion active parameters per token (8 of 128 experts fire per forward pass), quantised to Q4_K_M (approximately 19 GB on disk). Because this exceeds the host's 16 GB of physical RAM, Ollama serves the model through memory-mapped weight paging, loading pages of the model file into RAM only as the corresponding layer or expert is touched. This is the configuration used for the case-study evaluation reported in Chapter 4.
- `qwen2.5-coder:7b` - fully dense architecture, fits entirely within host RAM without paging, validated as a lower-latency alternative configuration.

Embedding model. `nomic-embed-text`, served locally via Ollama, producing 768-dimensional embeddings used for retrieval-augmented generation.

Sampling configuration. Temperature 0.05 for analysis-stage prompts, 0.2 for code-generation prompts, and 0.15 for test-generation prompts, as configured in `config.yaml`.

Vector store. ChromaDB, running locally with a SQLite-backed persistence layer, queried by cosine similarity with a 0.70 minimum-similarity retrieval threshold.

Appendix III

List of Software Components

Table III.1 lists the software libraries and tools used to build and run ALMS, corresponding to the technology stack introduced in Table 3.1.

Table III.1: Software Components and Libraries.

Component	Role
LangGraph	Stateful agent orchestration graph, fan-out and compile-retry cycles
LangChain Core	Underlying agent/LLM abstraction layer used by LangGraph
Ollama	Local large-language-model inference server
ChromaDB	Local vector database for retrieval-augmented generation
Python ast module	Standard-library abstract syntax tree parser used for static analysis
NetworkX	Graph data structure and algorithms library
python-louvain	Louvain community-detection implementation
Pydantic (v2)	Schema validation for inter-agent data contracts
Jinja2	Templating for generated code scaffolding
Black	Automatic Python code formatter
isort	Automatic Python import-statement sorter
py_compile	Standard-library Python bytecode compilation check
Bandit	Optional static security scanner for generated code
pytest	Test runner for generated test suites
Hypothesis	Property-based testing library
FastAPI	Target web framework for generated microservices
SQLAlchemy	Object-relational mapper for generated data-access code
Docker / Docker Compose	Containerisation and multi-service orchestration for generated stacks
Nginx	API gateway fronting the generated container stack
DiskCache	Local disk-backed cache for agent outputs, keyed on codebase hash
SQLite	Embedded database backing the audit log and vector store persistence
Rich	Console output formatting for the command-line interface

Appendix IV

List of Papers Presented and Published

No paper describing ALMS has, at the time of writing this report, been formally presented at a conference or published in a peer-reviewed venue. A manuscript describing the system and the case-study evaluation reported in Chapter 4 of this report has been drafted and is being prepared for submission to venues appropriate to a reproducible-systems and software-engineering-tools contribution, including *PeerJ Computer Science*, *SoftwareX*, *Information and Software Technology*, *Applied Sciences* (MDPI), and the *Journal of Systems and Software*; final formatting will follow the target venue’s requirements at the time of submission. This report will be updated, and the submission acknowledged accordingly, once the manuscript’s review outcome is known.